

Product Configurator API

Document Revision Log

Rev	Date	Name	Comment
A	--	--	Initial version
B	2021-04-22	--	Updated with example snippet
C	2021-05-11	--	
D	2022-02-09		Added the steps to deallocate variability data.
E	2022-12-01	--	Added section how to use Workflow handlers to import/export
F	2024-03-21		Added ITKs published in TC2312.0002 patch, Added description of custom exit to validate variant expression set that is promoted in TC2312.0002 patch
G	2024-10-24	Devendra Otari	Added Export large data set in different islands using custom closure rule
H	2024-11-13	--	

COPYRIGHT STATEMENT

Copyright in this work is vested in Products within Siemens Digital Industry Software.
All information contained in this document is the sole property of the Products Division.
Reproduction of this material in whole or in part is forbidden without the express consent of Products.

If printed, it is an uncontrolled copy.

Contents

Document Revision Log	1
Configurator data model	4
Common properties.....	4
Cfg0AbsConfiguratorWSO – Configurator workspace object	5
Cfg0AbsConfiguratorWSO Properties	5
Cfg0AbsConfiguratorWSO hierarchy.....	6
Cfg0FamilyGroup – Family Group.....	7
Required properties	7
Thread type	7
How to create Family and allocate?	7
Cfg0Allocation - Configurator Allocation.....	7
Why to create allocation objects?	8
Required properties	8
Thread type	8
Get allocations	9
Deallocate Features and Families	9
Cfg0AbsFamily - Family	10
The persistent BO for family	11
How to create Family and allocate?	11
Configurator Context (Cfg0ProductItem).....	13
Create the Cfg0ProductItem.....	13
Sample snippet.....	13
Make the feature available to a model.	13
Creating a sample Configurator Context data example	14
Add variant formula to a BOM line.	22
Create SVR (not Variant Criteria) using the configurator context.....	22
Configure using Variant Rule	22
CFG0_set_configuration_information	22
CFG0_get_configuration_information.....	25
Apply SVR to a BOM window.	26
Create and set a Boolean expression.....	26
Sample code for CFG0_analyze_configuration	28

Custom user exit to validate the expression on constraint object	30
Registering Custom User Exit.....	31
Example custom user exit	31
Sample JSON formula string	32
Sample JSON parser	33
Compute the available variability for a set of requested families which satisfies the input expression .	34
getAvailableVariability API	34
getAvailableVariability API sample	35
Sample input solveprofile	35
Retrieve the Configurator rules for given configurator perspective and set of criteria.....	37
Search constraints based on Features - Example	38
Sample Code	39
Customization Scenarios.....	39
Sample code to find referenced features, families for a given rule object (Cfg0AbsRule)	39
Sample code to find the features, descriptions for each literal in the variant formula of a given SVR	41
Using the workflow action handlers to export the rules in CSV dataset.....	55
Using the workflow action handlers to import the rules from CSV dataset	56
Export large data set in different islands using custom closure rule	57
ITK References	58
Boolean Solve:	58
Cfg0configurator.h.....	58

Configurator data model

Common properties

All business objects have the following common properties

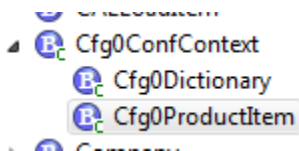
Property Name	Type	Storage Type	Inherited	Source	COTS
object_application	Attribute	String[32]		WorkspaceObject	✓
object_desc	Attribute	String[240]		WorkspaceObject	✓
object_name	Attribute	String[128]		WorkspaceObject	✓
object_properties	Attribute	Short	✓	POM_object	✓
object_string	Runtime	String[4000]	✓	BusinessObject	✓
object_type	Attribute	String[32]		WorkspaceObject	✓

Cfg0ProductItem – configurator context

It is a configurator context that manages variant options and rules for a specific configurable product. User can create a Cfg0ProductItem and associate the product specific configuration data to it.

For example, variant option DoorType such as 3-Door, 2-Door can be used only in a Refrigerator product and not in Air Conditioner product.

Hence, user can create and associate the DoorType option on a Cfg0ProductItem for Refrigerator product. Similarly, the constraint rules which are applicable to specific product are associated with Cfg0ProductItem. The product models are always specific to a product and hence, Cfg0ProductItem also anchors the product models. User can associate the Cfg0ProductItem to product content data such as CollaborativeDesign to consume the configuration data in product content parts in terms of variant conditions and rules



Important properties

Business Object : Cfg0ProductItem

Main Properties Operations Display Rules Deep Copy Rules Alternate ID Rules GRM Rules Operation Descriptor						
cfg0						
Property Name	Type	Storage Type	Inherited	Source	COTS	Referenced Type
cfg0PosBiasedVariantAvail	Attribute	Boolean		Cfg0ProductItem		
cfg0ConfigPerspective	Runtime	TypedReference	✓	Item		Cfg0Configur...

Cfg0AbsConfiguratorWSO – Configurator workspace object

Business Object : Cfg0AbsConfiguratorWSO

Main Properties Operations Display Rules Deep Copy Rules GRM Rules Operation Descriptor	
Details	
Project:	Cfg0configurator
Name	Cfg0AbsConfiguratorWSO
Display Name	Configurator Workspace Object
Storage Class	Cfg0AbsConfiguratorWSO
Parent	WorkspaceObject
Icon	Default
Type	Persistent
	☒ Is Abstract?
	☒ Is Exportable?
	☐ Allow creating instances of Secondary Business Objects

Cfg0AbsConfiguratorWSO Properties

Restricted

Main	Properties	Operations	Display Rules	Deep Copy Rules	GRM Rules	Operation Descriptor	
Enter search text here							
Property Name	Type	Storage Type	Inherited	Source	COT		
cfg0Sequence	Attribute	Integer		Cfg0AbsConfigurator			
cfg0Intents	Attribute	String[128]		Cfg0AbsConfigurator			
cfg0Effectivity	Runtime	String[4000]		Cfg0AbsConfigurator			
cfg0ObjectId	Compound	String[128]		Cfg0AbsConfigurator			
object name	Attribute	String[128]		WorkspaceObject			

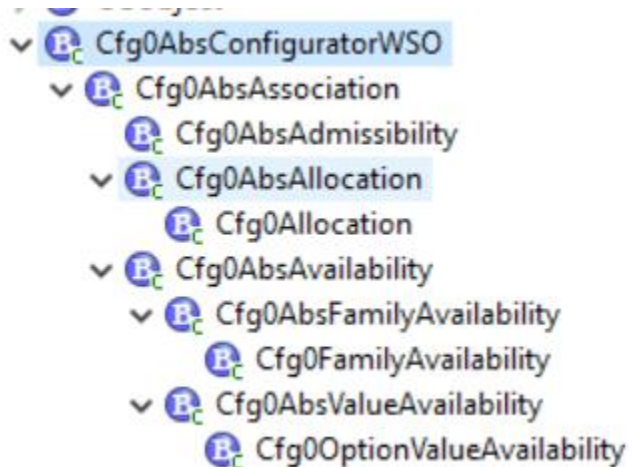
Name: *

Display Name: *

Description: *
☒ ReadOnly

Path: *

Cfg0AbsConfiguratorWSO hierarchy



Restricted

Cfg0FamilyGroup – Family Group

It acts as a container of variant configuration families of type Cfg0AbsFamily. The Cfg0FamilyGroup is created and managed in Product Configurator Application. User cannot create a subtype of Cfg0FamilyGroup.

Required properties

- Cfg0ObjectId
- object_name
- wso_thread

Thread type

Cfg0ThreadType is Cfg0FamilyGroupThread

How to create Family and allocate?

```
tag_t familyType;
ResultStatus stat = TCTYPE_ask_type ("Cfg0LiteralValueFamily", & familyType);
tag_t familyCreateInputTag;
stat = TCTYPE_construct_create_input( familyType, & familyCreateInputTag);
/* Set all the required properties - cfg0HasFreeFormValues,
cfg0IsDiscretionary, cfg0IsMultiSelect, cfg0ObjectId, cfg0ValueDataType,
object_name */

tag_t familyObj;
stat = TCTYPE_create_object( familyCreateInputTag, & familyObj);
stat = AOM_save_with_extensions( familyObj);
```

Cfg0Allocation - Configurator Allocation

Business Object : Cfg0Allocation

Main	Properties	Operations	Display Rules	Deep Copy Rules	GRM Rules	Operation D
Enter search text here						
Property Name	Type	Storage Type	Inherited			
• cfg0Protection	Attribute	Integer	✓			
• cfg0ProductItem	Compound	TypedReference	✓			
• cfg0Target	Runtime	TypedReference	✓			
• cfg0TargetObjectString	Compound	String[4000]	✓			
• cfg0Freeform	Compound	Boolean	✓			
• cfg0Sequence	Attribute	Integer	✓			
• cfg0Intents	Attribute	String[128]	✓			
• cfg0Effectivity	Runtime	String[4000]	✓			
• cfg0ObjectId	Compound	String[128]	✓			
• object_name	Attribute	String[128]	✓			

Why to create allocation objects?

Cfg0Allocation associates product configuration data such as Family or feature to a Cfg0ConfContext Item. The Cfg0Allocation helps user to define the owning Cfg0ConfContext Item for product configuration data. It also helps to reuse the same product configurator Family or feature across multiple products.

In user interface, for a given Cfg0ConfContext Item, a product configurator option object is configured its Cfg0Allocation object is configured in.

Required properties

- cfg0ObjectId
- cfg0ProductItem
- cfg0Target - Specifies revision of a Cfg0AbsConfiguratorWSO object which is associated with a Cfg0ConfContext or Cfg0AbsModel.
- object_name
- wso_thread

Thread type

- Cfg0ThreadType is Cfg0AllocationThread

Restricted

Get allocations

Given the configurator perspective object and specified objects such as Family, feature tags how do we get the allocation objects.

```
extern int CFG0CONFIGURATOR_API CFG0_get_allocation_objects(  
    const tag_t perspective,  
    /**< (I) The perspective of a Configurator Context which holds configuration  
    parameters such as Revision Rule, Rule Date etc.*/  
  
    const int n_objects,    /**< (I) Number of elements in @p objects.*/  
    const tag_t * objects,  
    /**< (I) n_objects List of object tags for which allocation objects are  
    requested.*/  
  
    int * n_allocations,    /**< (O) Number of elements in @p allocations.*/  
    CFG0_allocation_t ** allocations  
    /**< (OF) n_allocations Array of allocation revision .  
    <br/>The array is packed, which means that the memory  
    is to be deallocated by calling #MEM_free on the container only.*/  
);
```

Retrieves allocation objects for given configurator perspective and specified objects such as option family, value tags.

Deallocate Features and Families

Steps to deallocate allocated objects (Groups, Family, Features, etc)

1. Get the allocation objects for the group, family or feature using the ITK [CFG0_get_allocation_objects](#)
For example: Get all the allocation objects for all families in a given perspective.

```
int noOfOpFamilies = 0;  
scoped_smptr< tag_t > optionFamilies;  
AOM_ask_value_tags( perspective, "cfg0OptionFamilies", &noOfOpFamilies, &optionFamilies  
);  
  
int noOfAllocations = 0;  
scoped_smptr< CFG0_allocation_t > allocations;  
CFG0_get_allocation_objects( perspective, noOfOpFamilies, optionFamilies.get(),  
    &noOfAllocations, &allocations );
```

2. For each allocated object delete the allocated object to deallocate groups, family, features using the AOM ITKs
 - a. [AOM bulk delete instances](#) (allocation objects)
 - b. [AOM delete](#) (allocation object)
 - c. [AOM delete with extensions](#) (allocation object)

While validating the de-allocating the object the system will find whether any dependent objects of the workspace object which is being deallocated from product item, are allocated to given product item. If no dependent objects are allocated to the given product item, delete the allocation record for this workspace object. If dependent objects are allocated to the given product item, throw error indicating that given target WSO cannot be deallocated since its dependent objects are still allocated to same product item.

Deallocation will fail if :

- a. It is the last allocation revision being deleted AND
- b. The target object of allocation is also made available to model in given product context. We make a query to DB to find if there is any availability for target object.

While de-allocating logical family, its family's responsibility to de-allocate corresponding logical Feature from a given product item.

Cfg0AbsFamily - Family

The Cfg0AbsFamily represents a product variant configuration family which provides the variant configuration options to user.

For example, to provide different color options for his product say Red, Blue and Green, user can create a configuration family and associate it with the Cfg0ConfContext Item such as Cfg0ProductItem or Cfg0Dictionary.

Then, user can define the above Features linked to the family.

User can create the Cfg0AbsFamily with different data types such as Text, Integer, Floating Point, Boolean , etc

The information is stored in property cfg0ValueDataType (LOV Cfg0FamilyValueDataType) in Cfg0AbsFamilyThread object

with extra qualification such as

- Is Optional – cfg0IsDiscretionary

The following properties are part of the Cfg0AbsFamilyThread and required

- free-form values – cfg0HasFreeFormValues

Restricted

- multi-select values – cfg0IsMultiselect
- Unit Of Measure – cfg0UoM
- Family Namespace – cfg0FamilyNamespace

A family is uniquely identified with its attributes cfg0ObjectId and cfg0FamilyNamespace.

The persistent BO for family

- 1) Cfg0LiteralValueFamily (Thread - Cfg0LiteralFamilyThread)
- 2) Cfg0PackageOptionFamily (Cfg0PackageFamilyThread) - The Cfg0PackageOptionFamily is the concrete subtype of Cfg0AbsPackageOptionFamily. It serves as family for Feature Packages.
- 3) Cfg0SummaryOptionFamily (Thread - Cfg0SummaryFamilyThread) - The Cfg0SummaryOptionFamily is the concrete subtype of Cfg0AbsSummaryOptionFamily. It serves as family for Feature Summaries.

How to create Family and allocate?

```
tag_t familyType;
ResultStatus stat = TCTYPE_ask_type ("Cfg0LiteralValueFamily", & familyType);
tag_t familyCreateInputTag;
stat = TCTYPE_construct_create_input( familyType, & familyCreateInputTag);
/* Set all the required properties - cfg0HasFreeFormValues,
cfg0IsDiscretionary, cfg0IsMultiSelect, cfg0ObjectId, cfg0ValueDataType,
object_name */

tag_t familyObj;
stat = TCTYPE_create_object( familyCreateInputTag, & familyObj);
stat = AOM_save_with_extensions( familyObj);
```

Configurator Perspective (Cfg0ConfigPerspective)

Cfg0ConfiguratorPerspective is a runtime business object that represents the configuration information required for the Product Configurator in Teamcenter.

Cfg0ConfiguratorPerspective handles the **revision configuration behavior** of the allocated configurator objects.

It uses the **revision rule, the configurator context list; the compiled rule set date etc.** to determine the allocated and configured to be provided.

The cfg0ProductItems property is used to capture the configurator contexts.

cfg0RevisionRule property is used to capture the revision rule.

cfg0RuleSetCompileDate property is used to capture the date and time of the compiled rule set to be configured.

Cfg0ConfiguratorPerspective object is used as communication mechanism for the configuration and the valid configurator contexts across the Product Configurator and its consuming applications such as 4G Designer, Structure Manager.

Important Properties

Business Object : Cfg0ConfiguratorPerspective

Main Properties Operations Operation Descriptor						
cfg0						
Property Name	Type	Storage Type	Inherited	Source	COTS	Referenced Type
• cfg0DefaultRules	Runtime	TypedReference		• Cfg0ConfiguratorPers		• Cfg0AbsDefaultRule
• cfg0EditContext	Runtime	TypedReference		• Cfg0ConfiguratorPers		• BusinessObject
• cfg0ExcludeRules	Runtime	TypedReference		• Cfg0ConfiguratorPers		• Cfg0AbsExcludeRule
• cfg0FamilyGroups	Runtime	TypedReference		• Cfg0ConfiguratorPers		• Cfg0AbsFamilyGroup
• cfg0IncludeRules	Runtime	TypedReference		• Cfg0ConfiguratorPers		• Cfg0AbsIncludeRule
• cfg0Intents	Runtime	String[128]		• Cfg0ConfiguratorPers		
• cfg0ModelFamilies	Runtime	TypedReference		• Cfg0ConfiguratorPers		• Cfg0AbsModelFamily
...						
• cfg0Models	Runtime	TypedReference		• Cfg0ConfiguratorPers		• Cfg0AbsModel
• cfg0OptionFamilies	Runtime	TypedReference		• Cfg0ConfiguratorPers		• Cfg0AbsOptionFamily
• cfg0OptionValues	Runtime	TypedReference		• Cfg0ConfiguratorPers		• Cfg0AbsOptionValue
• cfg0ProductItems	Runtime	TypedReference		• Cfg0ConfiguratorPers		• Item
• cfg0RevisionRule	Runtime	TypedReference		• Cfg0ConfiguratorPers		• RevisionRule
• cfg0RuleSetCompileDate	Runtime	Date		• Cfg0ConfiguratorPers		
• cfg0RuleSetEffectivity	Runtime	String[4000]		• Cfg0ConfiguratorPers		
• cfg0SavedVariantRules	Runtime	TypedReference		• Cfg0ConfiguratorPers		• VariantRule
• cfg0VariantCriteria	Runtime	TypedReference		• Cfg0ConfiguratorPers		• VariantRule

Configurator Context (Cfg0ProductItem)

Create the Cfg0ProductItem

- **cfg0PosBiasedVariantAvail** - TRUE means variant options are available unless explicit availability rules exist that make them unavailable. FALSE means variant options are unavailable unless explicit availability rules exist that make them available.
- **object_name**
- **item_id**
- **revision**

Sample snippet

```
tag_t itemType;
ResultStatus stat = TCTYPE_ask_type ( "Cfg0ProductItem", &itemType );
tag_t itemCreateInputTag;
stat = TCTYPE_construct_create_input( itemType, &itemCreateInputTag );
std::string name = "ProductItem0092991";
stat = AOM_set_value_string( itemCreateInputTag, "object_name", name.c_str() );
stat = AOM_set_value_string( itemCreateInputTag, "item_id", "92991" ); //
Some unique id
stat = AOM_set_value_logical( itemCreateInputTag, "cfg0PosBiasedVariantAvail", false ); // Or true
tag_t productItem;
stat = TCTYPE_create_object( itemCreateInputTag, &productItem );
stat = AOM_save_with_extensions( productItem );
```

Make the feature available to a model.

If the product context is negatively biased, we need to create availability rule: Cfg0AvailabilityRule to make a feature available to a Product model

Sample code snippet creates Availability Rule for a Black feature of Color Family made available to M1 module



```

// Create availability rule for M=M1 && Color=Black
std::string availabilityM1BlackID = "AvailabilityRuleM1Black_" + itemNamespace;
tag_t avblRuleM1BlackCreInputTag;
ifail = TCTYPE_construct_create_input( availabilityRuleType, &avblRuleM1BlackCreInputTag );
// populate with values
ifail = AOM_set_value_string( avblRuleM1BlackCreInputTag, "cfg0ObjectId",
availabilityM1BlackID.c_str() );
ifail = AOM_set_value_string( avblRuleM1BlackCreInputTag, "object_name",
availabilityM1BlackID.c_str() );
ifail = AOM_set_value_string( avblRuleM1BlackCreInputTag, "cfg0Message", "Availability Rule failed."
);
ifail = AOM_set_value_string( avblRuleM1BlackCreInputTag, "cfg0Severity", "Error" );
ifail = AOM_set_value_tags( avblRuleM1BlackCreInputTag, "cfg0ProductItems", 1, &productItems[0]
);

// create object and save
tag_t avblRuleM1BlackTag;
ifail = TCTYPE_create_object( avblRuleM1BlackCreInputTag, &avblRuleM1BlackTag );
ifail = AOM_save_with_extensions( avblRuleM1BlackTag );
ifail = AOM_refresh( avblRuleM1BlackTag, 0 );

// Now set the include rule expression, applicability and subject
ifail = AOM_lock( avblRuleM1BlackTag );
const std::string subjectFormula = "[" + itemNamespace + "]" + colorFamily + "=" + blackFeature;
ifail = AOM_set_value_string( avblRuleM1BlackTag, "cfg0SubjectCondition", subjectFormula.c_str() );
ifail = AOM_save_with_extensions( avblRuleM1BlackTag );
ifail = AOM_unlock( avblRuleM1BlackTag );
ifail = AOM_refresh( avblRuleM1BlackTag, 0 );
ifail = AOM_lock( avblRuleM1BlackTag );

const std::string applicabilityFormula = "[" + itemNamespace + "]" + modelFamily + "=" + m1Model;
ifail = AOM_set_value_string( avblRuleM1BlackTag, "cfg0ApplicabilityCondition",
applicabilityFormula.c_str() );
ifail = AOM_save_with_extensions( avblRuleM1BlackTag );
ifail = AOM_unlock( avblRuleM1BlackTag );
ifail = AOM_refresh( avblRuleM1BlackTag, 0 );

```

Creating a sample Configurator Context data example

The following is an example code snippet creates the below mentioned data

- ❖ Car Context (Negatively biased)
 - Families
 - Color

Restricted

- Black
- White
- Red
- Model Family
 - M1
 - M2
 - M2
- Include Rule
 - (Applicability) Model=M1 -> (Subject) Color=Black
- Make Black, White features available to M1 model

```
#include <base_utils/IFail.hxx>
#include <base_utils/TcResultStatus.hxx>

...

try
{
    ResultStatus ifail = ITK_ok;
    std::string typeOfProductItem = "Cfg0ProductItem";
    tag_t itemType;
    ifail = TCTYPE_ask_type( typeOfProductItem.c_str(), &itemType );
    tag_t itemCreateInputTag;
    ifail = TCTYPE_construct_create_input( itemType, &itemCreateInputTag );
    std::string name = "ProductItem0092991";
    AOM_set_value_string( itemCreateInputTag, "object_name", name.c_str() );
    AOM_set_value_string( itemCreateInputTag, "item_id", "" ); // Some unique id
    AOM_set_value_logical( itemCreateInputTag, "cfg0PosBiasedVariantAvail", false ); // Or true
    tag_t productItem;
    ifail = TCTYPE_create_object( itemCreateInputTag, &productItem );
    ifail = AOM_save_with_extensions( productItem );

    char * itemNamespaceP;
    ifail = AOM_ask_value_string( productItem, "fnd0VariantNamespace", &itemNamespaceP );
    const std::string itemNamespace( itemNamespaceP );

    // Create product model family
    const std::string typeOfModelFamily = "Cfg0ProductModelFamily";
    tag_t modelFamilyType;
    ifail = TCTYPE_ask_type( typeOfModelFamily.c_str(), &modelFamilyType );
    tag_t modelFamilyCreInputTag;
    ifail = TCTYPE_construct_create_input( modelFamilyType, &modelFamilyCreInputTag );

    const std::string modelFamily = "ModelFamily" + itemNamespace;
    ifail = AOM_set_value_string( modelFamilyCreInputTag, "object_name", modelFamily.c_str() );
    ifail = AOM_set_value_string( modelFamilyCreInputTag, "cfg0ObjectId", modelFamily.c_str() );
    ifail = AOM_set_value_logical( modelFamilyCreInputTag, "cfg0IsMultiselect", false );
    ifail = AOM_set_value_logical( modelFamilyCreInputTag, "cfg0HasFreeFormValues", false );
}
```

```

ifail = AOM_set_value_logical( modelFamilyCreInputTag, "cfg0IsDiscretionary", false );
ifail = AOM_set_value_string( modelFamilyCreInputTag, "cfg0ValueDataType", "String" );
ifail = AOM_set_value_string( modelFamilyCreInputTag, "cfg0FamilyNamespace", "" );
ifail = AOM_set_value_tag( modelFamilyCreInputTag, "cfg0OwningProductItem", productItem );
tag_t modelFamilyTag;
ifail = TCTYPE_create_object( modelFamilyCreInputTag, &modelFamilyTag );
ifail = AOM_save_with_extensions( modelFamilyTag );

// Create product models M1, M2 and M3
const std::string m1Model = "M1_" + itemNamespace;
tag_t modelType;
ifail = TCTYPE_ask_type( "Cfg0ProductModel", &modelType );
tag_t model1CreInputTag;
ifail = TCTYPE_construct_create_input( modelType, &model1CreInputTag );
ifail = AOM_set_value_string( model1CreInputTag, "object_name", m1Model.c_str() );
ifail = AOM_set_value_string( model1CreInputTag, "cfg0ObjectId", m1Model.c_str() );
ifail = AOM_set_value_tag( model1CreInputTag, "cfg0ModelFamily", modelFamilyTag );
tag_t model1Tag;
ifail = TCTYPE_create_object( model1CreInputTag, &model1Tag );
ifail = AOM_save_with_extensions( model1Tag );

// Model = M2
const std::string m2Model = "M2_" + itemNamespace;
tag_t model2CreInputTag;
ifail = TCTYPE_construct_create_input( modelType, &model2CreInputTag );
ifail = AOM_set_value_string( model2CreInputTag, "object_name", m2Model.c_str() );
ifail = AOM_set_value_string( model2CreInputTag, "cfg0ObjectId", m2Model.c_str() );
ifail = AOM_set_value_tag( model2CreInputTag, "cfg0ModelFamily", modelFamilyTag );
tag_t model2Tag;
ifail = TCTYPE_create_object( model2CreInputTag, &model2Tag );
ifail = AOM_save_with_extensions( model2Tag );

// Model = M3
const std::string m3Model = "M3_" + itemNamespace;
tag_t model3CreInputTag;
ifail = TCTYPE_construct_create_input( modelType, &model3CreInputTag );
ifail = AOM_set_value_string( model3CreInputTag, "object_name", m3Model.c_str() );
ifail = AOM_set_value_string( model3CreInputTag, "cfg0ObjectId", m3Model.c_str() );
ifail = AOM_set_value_tag( model3CreInputTag, "cfg0ModelFamily", modelFamilyTag );
tag_t model3Tag;
ifail = TCTYPE_create_object( model3CreInputTag, &model3Tag );
ifail = AOM_save_with_extensions( model3Tag );

// Family Color
const std::string colorFamily = "Color_" + itemNamespace;
tag_t familyType;
ifail = TCTYPE_ask_type( "Cfg0LiteralValueFamily", &familyType );
tag_t familyCreInputTag;

```



```

ifail = TCTYPE_construct_create_input( familyType, &familyCreInputTag );

ifail = AOM_set_value_string( familyCreInputTag, "object_name", "Color" );
ifail = AOM_set_value_string( familyCreInputTag, "cfg0ObjectId", colorFamily.c_str() );
ifail = AOM_set_value_logical( familyCreInputTag, "cfg0IsMultiselect", false );
ifail = AOM_set_value_logical( familyCreInputTag, "cfg0HasFreeFormValues", false );
ifail = AOM_set_value_logical( familyCreInputTag, "cfg0IsDiscretionary", false );
ifail = AOM_set_value_string( familyCreInputTag, "cfg0ValueDataType", "String" );
ifail = AOM_set_value_string( familyCreInputTag, "cfg0FamilyNamespace",
itemNamespace.c_str() );

tag_t productItems[] = { productItem };

// Allocate Family to Product Item
ifail = AOM_set_value_tags( familyCreInputTag, "cfg0ProductItems", 1, &productItems[0] );
tag_t familyColorTag;
ifail = TCTYPE_create_object( familyCreInputTag, &familyColorTag );
ifail = AOM_save_with_extensions( familyColorTag );

tag_t allocationType;
ifail = TCTYPE_ask_type( "Cfg0Allocation", &allocationType );
// Allocate Family to Product Item

tag_t colorFamAllCreInputTag;
ifail = TCTYPE_construct_create_input( allocationType, &colorFamAllCreInputTag );
ifail = AOM_set_value_tag( colorFamAllCreInputTag, "cfg0ProductItem", productItem );
ifail = AOM_set_value_tag( colorFamAllCreInputTag, "cfg0Target", familyColorTag );
tag_t colorAllTag;
ifail = TCTYPE_create_object( colorFamAllCreInputTag, &colorAllTag );
ifail = AOM_save_with_extensions( colorAllTag );

// Create family Group
const std::string groupName = "GroupID" + itemNamespace;
tag_t groupType;
ifail = TCTYPE_ask_type( "Cfg0FamilyGroup", &groupType );
tag_t groupCreInputTag;
ifail = TCTYPE_construct_create_input( groupType, &groupCreInputTag );
ifail = AOM_set_value_string( groupCreInputTag, "cfg0ObjectId", groupName.c_str() );
ifail = AOM_set_value_string( groupCreInputTag, "object_name", groupName.c_str() );
tag_t families[] = { familyColorTag };
ifail = AOM_set_value_tags( groupCreInputTag, "cfg0Families", 1, &families[0] );
ifail = AOM_set_value_tags( groupCreInputTag, "cfg0ProductItems", 1, &productItems[0] );
tag_t groupTag;
ifail = TCTYPE_create_object( groupCreInputTag, &groupTag );
ifail = AOM_save_with_extensions( groupTag );

//Allocate Group to product item
tag_t groupAllCreInputTag;

```

```

ifail = TCTYPE_construct_create_input( allocationType, &groupAllCreInputTag );
ifail = AOM_set_value_tag( groupAllCreInputTag, "cfg0ProductItem", productItem );
ifail = AOM_set_value_tag( groupAllCreInputTag, "cfg0Target", groupTag );
tag_t groupAllTag;
ifail = TCTYPE_create_object( groupAllCreInputTag, &groupAllTag );
ifail = AOM_save_with_extensions( groupAllTag );

// Create Features White, Black, Red
const std::string whiteFeature = "White_" + itemNamespace;
tag_t featureType;
ifail = TCTYPE_ask_type( "Cfg0LiteralOptionValue", &featureType );
tag_t whiteFeatureCreInputTag;
ifail = TCTYPE_construct_create_input( featureType, &whiteFeatureCreInputTag );
ifail = AOM_set_value_string( whiteFeatureCreInputTag, "cfg0ObjectId", whiteFeature.c_str() );
ifail = AOM_set_value_string( whiteFeatureCreInputTag, "object_name", whiteFeature.c_str() );
ifail = AOM_set_value_tag( whiteFeatureCreInputTag, "cfg0OptionFamily", familyColorTag );
// Allocate White Feature to Product Item
ifail = AOM_set_value_tags( whiteFeatureCreInputTag, "cfg0ProductItems", 1, &productItems[0]
);

tag_t whiteFeatureTag;
ifail = TCTYPE_create_object( whiteFeatureCreInputTag, &whiteFeatureTag );
ifail = AOM_save_with_extensions( whiteFeatureTag );

//Allocate White Feature to product item
tag_t whiteFeatureAllCreInputTag;
ifail = TCTYPE_construct_create_input( allocationType, &whiteFeatureAllCreInputTag );
ifail = AOM_set_value_tag( whiteFeatureAllCreInputTag, "cfg0ProductItem", productItem );
ifail = AOM_set_value_tag( whiteFeatureAllCreInputTag, "cfg0Target", whiteFeatureTag );
tag_t whiteFeatureAllTag;
ifail = TCTYPE_create_object( whiteFeatureAllCreInputTag, &whiteFeatureAllTag );
ifail = AOM_save_with_extensions( whiteFeatureAllTag );

// Create Feature Black
const std::string blackFeature = "Black_" + itemNamespace;
tag_t blackFeatureCreInputTag;
ifail = TCTYPE_construct_create_input( featureType, &blackFeatureCreInputTag );
ifail = AOM_set_value_string( blackFeatureCreInputTag, "cfg0ObjectId", blackFeature.c_str() );
ifail = AOM_set_value_string( blackFeatureCreInputTag, "object_name", blackFeature.c_str() );
ifail = AOM_set_value_tag( blackFeatureCreInputTag, "cfg0OptionFamily", familyColorTag );
// Allocate Black Feature to Product Item
ifail = AOM_set_value_tags( blackFeatureCreInputTag, "cfg0ProductItems", 1, &productItems[0]
);

tag_t blackFeatureTag;
ifail = TCTYPE_create_object( blackFeatureCreInputTag, &blackFeatureTag );
ifail = AOM_save_with_extensions( blackFeatureTag );

//Allocate Black Feature to product item
tag_t blackFeatureAllCreInputTag;

```

```

ifail = TCTYPE_construct_create_input( allocationType, &blackFeatureAllCreInputTag );
ifail = AOM_set_value_tag( blackFeatureAllCreInputTag, "cfg0ProductItem", productItem );
ifail = AOM_set_value_tag( blackFeatureAllCreInputTag, "cfg0Target", blackFeatureTag );
tag_t blackFeatureAllTag;
ifail = TCTYPE_create_object( blackFeatureAllCreInputTag, &blackFeatureAllTag );
ifail = AOM_save_with_extensions( blackFeatureAllTag );

// Create Feature Red
const std::string redFeature = "Red_" + itemNamespace;
tag_t redFeatureCreInputTag;
ifail = TCTYPE_construct_create_input( featureType, &redFeatureCreInputTag );
ifail = AOM_set_value_string( redFeatureCreInputTag, "cfg0ObjectId", redFeature.c_str() );
ifail = AOM_set_value_string( redFeatureCreInputTag, "object_name", redFeature.c_str() );
ifail = AOM_set_value_tag( redFeatureCreInputTag, "cfg0OptionFamily", familyColorTag );
// Allocate Red Feature to Product Item
ifail = AOM_set_value_tags( redFeatureCreInputTag, "cfg0ProductItems", 1, &productItems[0] );
tag_t redFeatureTag;
ifail = TCTYPE_create_object( redFeatureCreInputTag, &redFeatureTag );
ifail = AOM_save_with_extensions( redFeatureTag );

//Allocate Red Feature to product item
tag_t redFeatureAllCreInputTag;
ifail = TCTYPE_construct_create_input( allocationType, &redFeatureAllCreInputTag );
ifail = AOM_set_value_tag( redFeatureAllCreInputTag, "cfg0ProductItem", productItem );
ifail = AOM_set_value_tag( redFeatureAllCreInputTag, "cfg0Target", redFeatureTag );
tag_t redFeatureAllTag;
ifail = TCTYPE_create_object( redFeatureAllCreInputTag, &redFeatureAllTag );
ifail = AOM_save_with_extensions( redFeatureAllTag );

// Creating availability - since the product context is negatively biased, we need to create
availability rule: Cfg0AvailabilityRule
std::string availabilityID = "AvailabilityRuleM1Red_" + itemNamespace;
tag_t availabilityRuleType, createAvailabilityRuleTagWso;
ifail = TCTYPE_ask_type( "Cfg0AvailabilityRule", &availabilityRuleType );
ifail = TCTYPE_construct_create_input( availabilityRuleType, &createAvailabilityRuleTagWso );
// populate with values
ifail = AOM_set_value_string( createAvailabilityRuleTagWso, "cfg0ObjectId", availabilityID.c_str()
);
ifail = AOM_set_value_string( createAvailabilityRuleTagWso, "object_name", availabilityID.c_str()
);
ifail = AOM_set_value_string( createAvailabilityRuleTagWso, "cfg0Message", "Availability Rule
failed." );
ifail = AOM_set_value_string( createAvailabilityRuleTagWso, "cfg0Severity", "Error" );
ifail = AOM_set_value_tags( createAvailabilityRuleTagWso, "cfg0ProductItems", 1,
&productItems[0] );

// create object and save
tag_t arRuleTag;

```

```

        ifail = TCTYPE_create_object( createAvailabilityRuleTagWso, &arRuleTag );
        ifail = AOM_save_with_extensions( arRuleTag );
        ifail= AOM_refresh( arRuleTag, 0 );

// Now set the include rule expression, applicability and subject
        ifail = AOM_lock( arRuleTag );
        const std::string subjectFormula = "[" + itemNamespace + "]" + colorFamily + "=" + redFeature;
        ifail = AOM_set_value_string( arRuleTag, "cfg0SubjectCondition", subjectFormula.c_str() );
        ifail = AOM_save_with_extensions( arRuleTag );
        ifail = AOM_unlock( arRuleTag );
        ifail = AOM_lock( arRuleTag );
        ifail = AOM_refresh( arRuleTag, 0 );

        const std::string applicabilityFormula = "[" + itemNamespace + "]" + modelFamily + "=" +
m1Model;
        ifail = AOM_set_value_string( arRuleTag, "cfg0ApplicabilityCondition",
applicabilityFormula.c_str() );
        ifail = AOM_save_with_extensions( arRuleTag );
        ifail = AOM_unlock( arRuleTag );
        ifail = AOM_refresh( arRuleTag, 0 );

// Create availability rule for M=M1 && Color=Black
        std::string availabilityM1BlackID = "AvailabilityRuleM1Black_" + itemNamespace;
        tag_t avblRuleM1BlackCreInputTag;
        ifail = TCTYPE_construct_create_input( availabilityRuleType, &avblRuleM1BlackCreInputTag );
        // populate with values
        ifail = AOM_set_value_string( avblRuleM1BlackCreInputTag, "cfg0ObjectId",
availabilityM1BlackID.c_str() );
        ifail = AOM_set_value_string( avblRuleM1BlackCreInputTag, "object_name",
availabilityM1BlackID.c_str() );
        ifail = AOM_set_value_string( avblRuleM1BlackCreInputTag, "cfg0Message", "Availability Rule
failed." );
        ifail = AOM_set_value_string( avblRuleM1BlackCreInputTag, "cfg0Severity", "Error" );
        ifail = AOM_set_value_tags( avblRuleM1BlackCreInputTag, "cfg0ProductItems", 1,
&productItems[0] );

// create object and save
        tag_t avblRuleM1BlackTag;
        ifail = TCTYPE_create_object( avblRuleM1BlackCreInputTag, &avblRuleM1BlackTag );
        ifail = AOM_save_with_extensions( avblRuleM1BlackTag );
        ifail= AOM_refresh( avblRuleM1BlackTag, 0 );

// Now set the include rule expression, applicability and subject
        ifail = AOM_lock( avblRuleM1BlackTag );
        const std::string subjectFormula = "[" + itemNamespace + "]" + colorFamily + "=" + blackFeature;
        ifail = AOM_set_value_string( avblRuleM1BlackTag, "cfg0SubjectCondition",
subjectFormula.c_str() );
        ifail = AOM_save_with_extensions( avblRuleM1BlackTag );

```

```

        ifail = AOM_unlock( avblRuleM1BlackTag );
        ifail = AOM_refresh( avblRuleM1BlackTag, 0 );
        ifail = AOM_lock( avblRuleM1BlackTag );

        const std::string applicabilityFormula = "[" + itemNamespace + "]" + modelFamily + "=" +
m1Model;
        ifail = AOM_set_value_string( avblRuleM1BlackTag, "cfg0ApplicabilityCondition",
applicabilityFormula.c_str() );
        ifail = AOM_save_with_extensions( avblRuleM1BlackTag );
        ifail = AOM_unlock( avblRuleM1BlackTag );
        ifail = AOM_refresh( avblRuleM1BlackTag, 0 );

        //Similarly we can create availability rule for following
        // Model=M1 & Color=White
        // Model=M2 & Color=Red
        // Model=M2 & Color=Black
        // Model=M2 & Color=White
        // Model=M2 & Color=Red
        // Model=M2 & Color=Black
        // Model=M2 & Color=White
        //

        // Create include rule Model=M1 -> Color=Black. This is how all the other rules (Default and
exclude) are created.
        tag_t includeRuleType;
        ifail = TCTYPE_ask_type( "Cfg0IncludeRule", &includeRuleType );
        tag_t includeRuleCreInputTag;
        const std::string incRuleName = "M1IncludesBlack_" + itemNamespace;
        ifail = TCTYPE_construct_create_input( includeRuleType, &includeRuleCreInputTag );
        ifail = AOM_set_value_string( includeRuleCreInputTag, "cfg0ObjectId", incRuleName.c_str() );
        ifail = AOM_set_value_string( includeRuleCreInputTag, "object_name", incRuleName.c_str() );
        ifail = AOM_set_value_string( includeRuleCreInputTag, "cfg0Severity", "Error" );
        ifail = AOM_set_value_string( includeRuleCreInputTag, "cfg0Message", "M1 includes Black" );
        ifail = AOM_set_value_tags( includeRuleCreInputTag, "cfg0ProductItems", 1, &productItems[0] );
        tag_t includeRuleTag;
        ifail = TCTYPE_create_object( includeRuleCreInputTag, &includeRuleTag );
        ifail = AOM_save_with_extensions( includeRuleTag );

        // Now set the include rule expression, applicability and subject
        ifail = AOM_lock( includeRuleTag );
        const std::string subjectFormula = "[" + itemNamespace + "]" + colorFamily + "=" + blackFeature;
        ifail = AOM_set_value_string( includeRuleTag, "cfg0SubjectCondition", subjectFormula.c_str() );
        ifail = AOM_save_with_extensions( includeRuleTag );
        ifail = AOM_unlock( includeRuleTag );
        ifail = AOM_lock( includeRuleTag );

        const std::string applicabilityFormula = "[" + itemNamespace + "]" + modelFamily + "=" +
m1Model;

```

```

        ifail = AOM_set_value_string( includeRuleTag, "cfg0ApplicabilityCondition",
applicabilityFormula.c_str() );
        ifail = AOM_save_with_extensions( includeRuleTag );
        ifail = AOM_unlock( includeRuleTag );
    }
    catch( IFail& )
    {
        //Catch error and report here
    }
}

```

Add variant formula to a BOM line.

To add the variant formula (condition) to the BOMLine, set the bl_formula string property on the BOMLine tag using the AOM_set_value_string

```
AOM_set_value_string( bomLineTag, "bl_formula", "[NS]Engine = Petrol" )
```

Create SVR (not Variant Criteria) using the configurator context.

Use existing VRULE_create ITK.

For example:

```
tag_t svrObject = NULLTAG;
VRULE_create( "svrName", "svrDescription", &svrObject );
```

Note: The SVR can be attached to a Configurator Context (IMAN_Specifications) or it can be attached to the BOM Content (IMAN_Reference). The [GRM_create_relation](#) API can be used for the same.

Configure using Variant Rule

In Product Configurator, Variant Rules, SVR are created to be applied to a Product Structure to configure. From 11.6 release, the configuration information used for variant configuration can also be persisted with the SVR. The configuration information includes details such as Configurator context, reporting severity, revision rule information and other configuration profile information that influences how the configuration is solved. The list of profile tokens are defined [here](#).

CFG0_set_configuration_information

Sets the configuration information on a session configurable object. The session configurable object can be a configurator perspective or SVR objects.

The following is the sample snippet to set the order mode configuration information

```

int nBoolTypeProfileEntries = 10;
std::vector <CFG0_conf_solver_profile_bool_entry_t> boolTypeProfileEntries;

// Add Order mode keys
if(m_solveMode == PCA0_order)
{
    boolTypeProfileEntries = getOrderModeProfileKeys();
    nBoolTypeProfileEntries = nBoolTypeProfileEntries + boolTypeProfileEntries.size();
}
boolTypeProfileEntries.reserve( nBoolTypeProfileEntries );

CFG0_conf_solver_profile_bool_entry_t boolProfileEntry1;

boolProfileEntry1.key_text = "CFG_PROFILE_applyConfigConstraints";
boolProfileEntry1.n_values = 1;
boolProfileEntry1.values = ( bool* ) &booltrueValVec;
boolTypeProfileEntries.push_back( boolProfileEntry1 );

if( !isAnalyze )
{
    CFG0_conf_solver_profile_bool_entry_t boolProfileEntry2;

    boolProfileEntry2.key_text = "CFG_PROFILE_getAllAvailableValuesPerFamily";
    boolProfileEntry2.n_values = 1;
    boolProfileEntry2.values = ( bool* ) &booltrueValVec;
    boolTypeProfileEntries.push_back( boolProfileEntry2 );
}
else
{
    CFG0_conf_solver_profile_bool_entry_t skipSolutionSeverityProcessingEntry;

    skipSolutionSeverityProcessingEntry.key_text = "CFG_PROFILE_skipSolutionSeverityProcessing";
    skipSolutionSeverityProcessingEntry.n_values = 1;
    skipSolutionSeverityProcessingEntry.values = ( bool* ) &booltrueValVec;
    boolTypeProfileEntries.push_back( skipSolutionSeverityProcessingEntry );
}

CFG0_conf_solver_profile_bool_entry_t validateAndSkipViolationMinimizationEntry;

validateAndSkipViolationMinimizationEntry.key_text =
"CFG_PROFILE_validateAndSkipViolationMinimization";
validateAndSkipViolationMinimizationEntry.n_values = 1;
validateAndSkipViolationMinimizationEntry.values = ( bool* ) &booltrueValVec;
boolTypeProfileEntries.push_back( validateAndSkipViolationMinimizationEntry );

CFG0_conf_solver_profile_bool_entry_t boolProfileEntry3;

```

```

boolProfileEntry3.key_text = "CFG_PROFILE_reportViolationForMutuallyExclusiveConstraint";
boolProfileEntry3.n_values = 1;
boolProfileEntry3.values = ( bool* ) &booltrueValVec;
boolTypeProfileEntries.push_back( boolProfileEntry3 );

std::vector <CFG0_conf_solver_profile_int_entry_t> intTypeProfileEntries;
CFG0_conf_solver_profile_int_entry_t intProfileEntry1;

intProfileEntry1.key_text = "CFG_PROFILE_minErrorSeverity";
const int valueInts1[1] = { errorSeverity };
intProfileEntry1.values = &valueInts1[0];
intProfileEntry1.n_values = 1;
intTypeProfileEntries.push_back( intProfileEntry1 );

CFG0_conf_solver_profile_int_entry_t intProfileEntry2;

intProfileEntry2.key_text = "CFG_PROFILE_minReportSeverity";
const int valueInts2[1] = { reportSeverity };
intProfileEntry2.values = &valueInts2[0];
intProfileEntry2.n_values = 1;
intTypeProfileEntries.push_back( intProfileEntry2 );

CFG0_conf_solver_profile_info_t solverProfileInfo;
solverProfileInfo.n_int_type_profile_entries = intTypeProfileEntries.size();
solverProfileInfo.int_type_profile_entries = &intTypeProfileEntries[0];
solverProfileInfo.n_bool_type_profile_entries = boolTypeProfileEntries.size();
solverProfileInfo.bool_type_profile_entries = &boolTypeProfileEntries[0];

// All other types must be defaulted to 0
solverProfileInfo.n_str_type_profile_entries = 0;
solverProfileInfo.str_type_profile_entries = NULL;
solverProfileInfo.n_double_type_profile_entries = 0;
solverProfileInfo.double_type_profile_entries = NULL;
solverProfileInfo.n_tag_type_profile_entries = 0;
solverProfileInfo.tag_type_profile_entries = NULL;
solverProfileInfo.n_date_type_profile_entries = 0;
solverProfileInfo.date_type_profile_entries = NULL;

// configPerspective – Here the config Perspective tag can be obtained from the
// cfg0ConfigPerspective runtime property of Cfg0ProductItem
// AOM_ask_value_tag( configContext, "cfg0ConfigPerspective", &configPerspective )

ResultStatus stat = CFG0_set_configuration_information( configPerspective, &solverProfileInfo,
svtTag );

```



```

std::vector<CFG0_conf_solver_profile_bool_entry_t> getOrderModeProfileKeys( )
{

    CFG0_conf_solver_profile_bool_entry_t boolProfileEntry1;

    boolProfileEntry1.key_text = "CFG_PROFILE_preferEmptyValueForDiscretionary";
    boolProfileEntry1.n_values = 1;
    boolProfileEntry1.values = ( bool* ) &booltrueValVec;
    boolTypeProfileEntries.push_back(boolProfileEntry1);

    CFG0_conf_solver_profile_bool_entry_t boolProfileEntry2;

    boolProfileEntry2.key_text = "CFG_PROFILE_considerLowSeverityRulesForExpansion";

    boolProfileEntry2.n_values = 1;
    boolProfileEntry2.values = ( bool* ) &booltrueValVec;
    boolTypeProfileEntries.push_back( boolProfileEntry2 );
    Teamcenter::Pca0awConfiguratorLogger::logger()->debug( "setting
considerLowSeverityRulesForExpansion for configure action" );

    CFG0_conf_solver_profile_bool_entry_t boolProfileEntry3;

    boolProfileEntry3.key_text = "CFG_PROFILE_applyDefaults";

    boolProfileEntry3.n_values = 1;
    boolProfileEntry3.values = ( bool* ) &booltrueValVec;
    boolTypeProfileEntries.push_back( boolProfileEntry3 );

    return boolTypeProfileEntries;
}

```

CFG0_get_configuration_information

Gets the configuration information from the session configurable objects (SVR, etc).

Sample Snippet

```

int noOfSolverProfiles = 2;
scoped_smptr< CFG0_conf_solver_profile_info_t > solverProfileInfos;
scoped_smptr< tag_t > perspectives;
std::vector< tag_t > sessionConfigObjects;
sessionConfigObjects.push_back( variantRule1 );
sessionConfigObjects.push_back( variantRule2 );

```

```
ResultStatus stat = CFG0_get_configuration_information( noOfSolverProfiles,
&sessionConfigObjects.front(), &perspectives, &solverProfileInfos );
```

Apply SVR to a BOM window.

```
int BOM_window_apply_variant_configuration(
    tag_t window,  /**< (I) The BOM Window */
    int n_svr,      /**< (I) The number of Saved Variant Rules to apply to the BOM
Window. */
    const tag_t* svrObj  /**< (I) n_svr The Saved Variant Rules to apply to the BOM Window. */
)
```

Sets the given variant rules to be the current rules for the given BOM window and applies the configuration on BOM window.

It will override any existing configuration.

If no variant rule is passed, the default configuration will be applied on the window.

Apply

Create and set a Boolean expression

See reference here:

https://docs.plm.automation.siemens.com/data_services/resources/tc/12.2/tcrefs/custom/en_US/ITKFunction/group_boolean_solve.html#ga5ca5c95d4238e069d64fcf15dab586db

For example, a variant expression "([ProductCar]model = Luxury) & [ProductCar]color = Red" can be represented as:

```
// [ProductCar]model
BSLV_option_family_info_t model_family;
model_family.family_id = "model";
model_family.family_namespace = "ProductCar";
// Luxury
BSLV_option_value_info_t luxury_model;
luxury_model.family = &model_family;
luxury_model.value_id = "M1";
// [ProductCar]color
BSLV_option_family_info_t color_family;
color_family.family_id = "color";
color_family.family_namespace = "ProductCar";
// Red
BSLV_option_value_info_t color_red;
color_red.family = &color_family;
```

```

color_red.value_id = "Red";
// Construct the expression info
// [ProductCar]model = Luxury
BSLV_expression_info_t modelExpr;
modelExpr.operator_code = CFG_operator_is_equal;
modelExpr.value = &luxury_model;
modelExpr.left = 0;
modelExpr.right = 0;
// [ProductCar]color = Red
BSLV_expression_info_t colorFamilyExpr;
colorFamilyExpr.operator_code = CFG_operator_is_equal;
colorFamilyExpr.value = &color_red;
colorFamilyExpr.left = 0;
colorFamilyExpr.right = 0;
// ( [ProductCar]model = Luxury )
BSLV_expression_info_t bracesExpr;
bracesExpr.operator_code = CFG_operator_brackets;
bracesExpr.left = &modelExpr;
bracesExpr.right = 0;
// ( [ProductCar]model = Luxury ) & [ProductCar]color = Red
BSLV_expression_info_t finalExpr;
finalExpr.operator_code = CFG_operator_and;
finalExpr.value = 0;
finalExpr.left = &bracesExpr;
finalExpr.right = &colorFamilyExpr;

```

From variability, user can construct either a Teamcenter formula string e.g. [namespace]family = value or user can create a structured expression as follows:

struct **BSLV_expression_info**

```

{
    int operator_code;          /**< One of the operator tokens in <configurator/cfg_tokens.h> e.g.
#CFG_operator_is_equal */
    const BSLV_option_value_info_t* value; /**< NULL if @p opcode is a Boolean operator, e.g.
#CFG_operator_and */
    const BSLV_expression_info_t* left;    /**< NULL if @p opcode is a comparison operator, e.g.
#CFG_operator_is_equal */
    const BSLV_expression_info_t* right;   /**< NULL if @p opcode is a comparison operator or a unary
operator, e.g. #CFG_operator_brackets */
};
typedef struct
{
    const BSLV_option_family_info_t* family; /**< The option family info for @p value_id */
    const char* value_id;                    /**< The ID of a variant option value */
} BSLV_option_value_info_t;

```

Restricted

Sample code for CFG0_analyze_configuration

Note: Need the gateway license “**tc_ipd_config_gtwy**” to use this API



SampleCode.cpp

```
#include <Cfg0configurator/cfg0configurator.h>

/**
 * Holds a variant Feature information.
 */
typedef struct
{
    tag_t family;          /**< The Family object for @p value_id. */
    const char * value_id; /**< The ID of the Feature. */
    tag_t value;           /**< The object of the Feature. */
} CFG0_option_value_info_t;

/**
 * Qualifies the Features associated with an expression. The expression could be obtained from evaluation of a configuration
 * against a configuration. The expression will have a number of values which can be mapped to the contributing constraints or
 * the input configuration which can be treated as a user constraint.
 */
typedef struct
{
    CFG_severity_t severity;          /**< The severity of the contributing constraint. */
    const CFG0_option_value_info_t* value_info; /**< The contributed Feature information. */
} CFG0_sample_expr_value_info_t;

void analyzeConfig()
{
    AOM_ask_value_tag( m_contextItem, "cfg0ConfigPerspective", &perspective )

    const bool booltrueValVec [ 1 ] = { true };

    std::vector< CFG0_conf_solver_profile_bool_entry_t > boolTypeProfileEntries;

    bool isMinErrorSeveritySet = false;

    // CFG_PROFILE_report_violtn_for_mutually_excl_constraint_key
    CFG0_conf_solver_profile_bool_entry_t reportVoilaitionForMEconstrProfileEntry;
    const char *applyConfigCnstProfileEntryKeyText = CFG_PROFILE_report_violtn_for_mutually_excl_constraint_key;
    reportVoilaitionForMEconstrProfileEntry.key_text = applyConfigCnstProfileEntryKeyText;
    reportVoilaitionForMEconstrProfileEntry.n_values = 1;
    reportVoilaitionForMEconstrProfileEntry.values = (bool*) &booltrueValVec;
    boolTypeProfileEntries.push_back( reportVoilaitionForMEconstrProfileEntry );

    // CFG_PROFILE_prefer_empty_value_for_discretionary_family_key
    CFG0_conf_solver_profile_bool_entry_t preferEmptyValforDiscretionaryProfileEntry;
    const char *applyConfigCnstProfileEntryKeyText = CFG_PROFILE_prefer_empty_value_for_discretionary_family_key;
    preferEmptyValforDiscretionaryProfileEntry.key_text = applyConfigCnstProfileEntryKeyText;
    preferEmptyValforDiscretionaryProfileEntry.n_values = 1;
    preferEmptyValforDiscretionaryProfileEntry.values = (bool*) &booltrueValVec;
    boolTypeProfileEntries.push_back( preferEmptyValforDiscretionaryProfileEntry );

    // CFG_PROFILE_consider_low_severity_rules_for_expansion_key
    CFG0_conf_solver_profile_bool_entry_t considerLowSeverityRulesProfileEntry;
    const char *applyConfigCnstProfileEntryKeyText = CFG_PROFILE_consider_low_severity_rules_for_expansion_key;
    considerLowSeverityRulesProfileEntry.key_text = applyConfigCnstProfileEntryKeyText;
```

Restricted

```

considerLowSeverityRulesProfileEntry.n_values = 1;
considerLowSeverityRulesProfileEntry.values = (bool*) &booltrueValVec;
boolTypeProfileEntries.push_back( considerLowSeverityRulesProfileEntry );

CFG0_conf_solver_profile_bool_entry_t applyConfigCnstProfileEntry;
const char *applyConfigCnstProfileEntryKeyText = CFG_PROFILE_apply_config_constraints_key;
applyConfigCnstProfileEntry.key_text = applyConfigCnstProfileEntryKeyText;
applyConfigCnstProfileEntry.n_values = 1;
applyConfigCnstProfileEntry.values = (bool*) &booltrueValVec;
boolTypeProfileEntries.push_back( applyConfigCnstProfileEntry );

CFG0_conf_solver_profile_bool_entry_t applyDefaultsProfileEntry;
const char *applyDefaultsProfileEntryKeyText = CFG_PROFILE_apply_defaults_key;
applyDefaultsProfileEntry.key_text = applyDefaultsProfileEntryKeyText;
applyDefaultsProfileEntry.n_values = 1;
applyDefaultsProfileEntry.values = (bool*) &booltrueValVec;
boolTypeProfileEntries.push_back( applyDefaultsProfileEntry );

CFG0_conf_solver_profile_bool_entry_t allAvailableValsProfileEntry;
const char *applyDefaultsProfileEntryKeyText = CFG_PROFILE_get_all_available_values_per_family_key;
allAvailableValsProfileEntry.key_text = applyDefaultsProfileEntryKeyText;
allAvailableValsProfileEntry.n_values = 1;
allAvailableValsProfileEntry.values = (bool*) &booltrueValVec;
boolTypeProfileEntries.push_back( allAvailableValsProfileEntry );

const int value_ints [ 1 ] = { (int) CFG_severity_error_level };
const char *minErrorSeverityKeyText = CFG_PROFILE_min_error_severity_key;
CFG0_conf_solver_profile_int_entry_t minErrorSeverity;
minErrorSeverity.key_text = minErrorSeverityKeyText;
minErrorSeverity.n_values = 1;
minErrorSeverity.values = (int*) &value_ints;
intTypeProfileEntries.push_back( minErrorSeverity );

CFG0_conf_solver_profile_info_t solverProfileInfo;

solverProfileInfo.n_int_type_profile_entries = intTypeProfileEntries.size();
solverProfileInfo.int_type_profile_entries = &intTypeProfileEntries [ 0 ];

solverProfileInfo.n_bool_type_profile_entries = boolTypeProfileEntries.size();
solverProfileInfo.bool_type_profile_entries = &boolTypeProfileEntries [ 0 ];

// All other types must be defaulted to 0
solverProfileInfo.n_str_type_profile_entries = 0;
solverProfileInfo.str_type_profile_entries = NULL;
solverProfileInfo.n_double_type_profile_entries = 0;
solverProfileInfo.double_type_profile_entries = NULL;
solverProfileInfo.n_tag_type_profile_entries = 0;
solverProfileInfo.tag_type_profile_entries = NULL;
solverProfileInfo.n_date_type_profile_entries = 0;
solverProfileInfo.date_type_profile_entries = NULL;
CFG0_set_configuration_information( perspective, &solverProfileInfo, perspective );

const char* in_expr = "[NS]Models = LXI & [NS]A = A1";

CFG_satisfiability satisfiable;
char* sample_expr;
int n_sample_value_states = 0;
CFG0_sample_expr_value_info_t * sample_value_states;
int n_conflicts = 0;
CFG0_conflict_info_t * conflicts;
logical complete;
int n_required_families = 0;
tag_t * required_families;

CFG0_analyze_configuration
(
    perspective,
    in_expr,
    0,
    &satisfiable,
    &sample_expr,
    &n_sample_value_states,
    &sample_value_states,
    &n_conflicts,

```

```

    &conflicts,
    &complete,
    &n_required_families,
    &required_families
);

for ( int inx = 0; inx < n_sample_value_states; ++inx )
{
    CFG0_sample_expr_value_info_t & value_state = sample_value_states[ inx ];
    const CFG0_option_value_info_t & optionValue = *( value_state.value_info );

    std::string actualValueObjId;
    actualValueObjId = optionValue.value_id; // Feature ID
    // optionValue.value = Value Tag
    // optionValue.family = Family Tag
    // value_state.severity = Contains severity.

    /*
    The values of the severity
    typedef enum CFG_severity
    {
        CFG_severity_init_empty = -8,    // < A constraint has initialized a family with the empty value.
        CFG_severity_default_selected = -4, // < A constraint has assigned a default value.
        CFG_severity_package_default = -2, // < A constraint has assigned a default value.
        CFG_severity_unknown = 0,        // < An administrator authored constraint with unknown severity has assigned a value.
        CFG_severity_information_level = 1, // < A constraint with INFORMATION severity has assigned a value.
        CFG_severity_warning_level = 2,    // < A constraint with WARNING severity has assigned a value.
        CFG_severity_error_level = 4,      // < A constraint with ERROR severity has assigned a value.
        CFG_severity_input_selected = 8    // < The value was specified in the input.
    } CFG_severity_t;

    */

    char * actualValueId;
    AOM_ask_value_string( optionValue.value, "cfg0ObjectId", &actualValueId )
    actualValueObjId = actualValueId;
    char * actualFamilyId;
    AOM_ask_value_string( optionValue.family, "cfg0ObjectId", &actualFamilyId )
}

}

```

Custom user exit to validate the expression on constraint object

1. An extension point for customers to plug the validation while setting the expression on a configurable object. The operation will not set the expression on the configurable object if any of the expression is invalid as per this user exit.
2. When user exit is overridden or defined it will be called before any expression is being set. The user exit will be provided with the input expression in the JSON string format of the expression along with the UID of the configurable object.
3. The user exit is expected to return non-zero value representing the custom ifail ID. If it returns ITK_ok then the input expression string is valid.
4. Available from release TC2312.0002 patch

Registering Custom User Exit

Register user exit "USER_BSLV_validate_set_expression" with the custom library and function name.

Example: Library: libcustconfig, function name: CUST_validate_set_expression

```
extern "C" DllExport int libcustconfig_register_callbacks()
{
    CUSTOM_register_exit( "libcustconfig", "USER_BSLV_validate_set_expression",
                        (CUSTOM_EXIT_ftn_t) CUST_validate_set_expression );
    return 0;
}
```

- For the custom library libcustconfig, define an extern "C" function libcustconfig_register_callbacks(). This function is called by the Teamcenter framework when the libcustconfig library is loaded by the tcserver.
- CUSTOM_register_exit("libcustconfig", "USER_BSLV_validate_set_expression", (CUSTOM_EXIT_ftn_t) CUST_validate_set_expression);
 - The "libcustconfig" is the name of the custom library.
 - In this example, CUST_validate_set_expression is the function that is registered for the custom exit 'USER_BSLV_validate_set_expression'
 - The library libcustconfig should be added to the preference "TC_customization_libraries" so that it can be loaded by the tcserver.
 - Command to set the preference value: **preferences_manager** -mode=import -u=<USER> -p=<Pwd> -g=<grp> -preference=TC_customization_libraries -values=libcustconfig -action=OVERRIDE -scope=SITE

Example custom user exit

```
extern int CUST_validate_set_expression( METHOD_message_t* /*msg*/, va_list args )
{
    // First argument is the constraint object
    tag_t varConfig = va_arg ( args, tag_t );
    // Second argument is the Expression to be set in the JSON format
    std::string tcFormulaJSON = va_arg ( args, char * );

    // Parse the JSON formula. The sample code provided in the kit has the example
    // parser that parses the formula to a C++ struct
}
```

```

// Perform the custom validation

If( valid )
{
    return ITK_ok;
}
else
{
    std::string errorMsg << " ... "; // Add the appropriate custom error message
    throw IFail( CFG_invalid_expression, errorMsg.c_str() );
}
}

```

Sample JSON formula string

For the expressionType valid values refer to cfg_tokens.h file under [booleanmath/cfg_tokens.h](#)

```

{
  "applicationConfigExpression": [
    {
      "expressionType": 28, // 28 = include rule.
      "configExpressionSet": [
        {
          "configExpressionSections": [
            {
              "expressionType": 25, // 25 = condition expression
              "subExpressions": [
                {
                  "expressionGroups": [
                    {
                      "groupName": "[000023]Models-000023",
                      "terms": [
                        {
                          "family": "AYiVss7SAAgcRA",
                          "familyId": "Models-000023",
                          "familyNamespace": "000023",
                          "operatorCode": 5,
                          "feature": "AYsVss7SAAgcRA",
                          "valueText": "LXI-000023",
                          "severity": 8
                        }
                      ]
                    }
                  ]
                }
              ]
            }
          ]
        },
        {
          "expressionType": 44, // 44 = subject expression
          "subExpressions": [
            {
              "expressionGroups": [
                // each element under expression group is ANDed

```

Restricted


```

    { //
      "groupName": "[000023]Engine-000023",
      "terms": [
        // Each element under terms is ORed
        {
          "family": "AwiVss7SAAgcRA",
          "familyId": "Engine-000023",
          "familyNamespace": "000023",
          "operatorCode": 5,
          "feature": "A0uVss7SAAgcRA",
          "valueText": "Diesel",
          "severity": 8
        }
      ]
    }
  ]
}

```

Sample JSON parser

Please find the sample parser implementation to parse the expression JSON to the appropriate structure under `sample/Cfg0configurator/sample/customlib`

[*JSONGrid.hxx*](#)

Header file providing definition of public functions to covert JSON string to C++ structs.

[*JSONGrid.cxx*](#)

This file contains the implementation of public functions to convert JSON string to C++ structs.

The schema is defined in `$TC_DATA/json/configurator/schema/CFG0_configurator_definitions.json` as part of `ApplicationConfigExpression` object.

[*Converting JSON string to structs*](#)

Please make sure you include `JSONGrid.hxx` and `JSONGrid.cxx` in your source code.

Include following files in the custom implementation.

Please make sure the `base_utils/TcRapidJson.hxx` and `rapidjson/*.h` files are in include path.

Restricted

```

#include <custdir/JSONGrid.hxx>
...
...
extern int CUST_validate_set_expression( METHOD_message_t* /*msg*/, va_list args )
{
    // First argument is the constraint object
    tag_t varConfig = va_arg ( args, tag_t );
    // Second argument is the Expression to be set in the JSON format
    std::string tcFormulaJSON = va_arg ( args, char * );

    rapidjson::Document josnDocInput;
    josnDocInput.Parse( tcFormulaJSON.c_str() );

    const rapidjson::Value& jsonGrid = josnDocInput["applicationConfigExpression"];
    if ( !jsonGrid.IsArray() || jsonGrid.Size() != 1 )
    {
        return CFG_invalid_expression;
    }

    // Get the PCAGrid from ApplicationConfigExpression
    Teamcenter::JSONGrid::ApplicationConfigExpression pcaGrid;
    Teamcenter::JSONGrid::createApplConfigExpressionFromValue( jsonGrid[0], pcaGrid );
    // jsonGrid vector will always have one member

    if ( !pcaGrid.configExprSets.empty() )
    {
        ...
        // Custom validation behavior
    }
    ...
    return ITK_ok; // Or return appropriate ifail
}

```

Compute the available variability for a set of requested families which satisfies the input expression

1. This section defines the published API that computes the available variability for a set of requested families which satisfies the input expression against the constraint(s) present in the system. Available from release TC2312.0002 patch

getAvailableVariability API

The API Computes the available variability for a set of requested families which satisfies the input expression against the constraint(s) present in the system. The operation allows only precise expression to be passed as input to retrieve the available variability for the requested families.

Restricted

Returns

- ITK_ok on success
- CFG0CONFIGURATOR_invalid_configuration_perspective if input perspective is invalid.
- CFG_invalid_solve_profile if input solveProfileJson is an empty or invalid string.
- CFG_invalid_solve_profile_duplicate_keys if input solveProfileJson contains duplicate keys.
- CFG0CONFIGURATOR_JSON_schema_validation_failed if input solveProfileJson fails schema validation.
- CFG_convert_expr_invalid_input if inputExpression contains syntax errors.
- CFG0CONFIGURATOR_invalid_family_count if input families is empty.

```
CFG0CONFIGURATOR_API extern int getAvailableVariability(  
  
    tag_t perspective,                                     /**< (I) The perspective of a Configurator Context  
                                                         which holds configuration parameters such as Revision  
                                                         Rule, Rule Date, Product Hierarchy Depth etc. */  
  
    const std::string& solveProfileJson,                 /**< (I) A solve operation profile as JSON strings that  
                                                         consists of attributes governing the behavior of  
                                                         configurator constraint evaluation. */  
  
    const ConfigurableExpression& inputExpression,       /**< (I) The input configuration expression to be  
                                                         analyzed. */  
  
    const std::vector<tag_t>& families,                   /**< (I) Families for which availability expressions will  
                                                         be provided. */  
  
    std::vector<AvailableVariability>& availableVariability, /**< (O) A list of available variability for the families of  
                                                         interest. An entry in the list contains a requested family  
                                                         along with its available features or range expressions.  
                                                         An input family with no valid values will contain an  
                                                         element of available variability with zero expression  
                                                         terms. The list will be empty in case of invalid  
                                                         configuration. */  
  
    std::string& expandedExpression,                     /**< (O) The expanded expression which is a formula  
                                                         string satisfying the given input expression and set of  
                                                         constraint(s) for the given configurator context.*/  
  
    bool& isValid,                                       /**< (O) The result of analysis whether the @p  
                                                         inputExpression is valid. */  
  
    bool& isComplete                                    /**< (O) The result of analysis whether the @p  
                                                         inputExpression is complete. */  
);
```

getAvailableVariability API sample

The sample code to use the API is provided under `getAvailableVariability_sample.cxx` file

Sample input solveprofile

Restricted

The solve operation profile as JSON strings that consists of attributes governing the behaviour of configurator constraint evaluation.

The string adheres to a JSON schema for solve operation profile defined in `SolveProfile_Schema.json`. Refer to the schema defined at `@TC_DATA/json/variabilityadaptor/schema` for details.

Sample Solve profile

```
{
  "solveProfile": [
    {
      "key": "CFG_PROFILE_applyConfigConstraints",
      "value": true
    },
    {
      "key": "CFG_PROFILE_check_completeness",
      "value": true
    },
    {
      "key": "CFG_PROFILE_minErrorSeverity",
      "value": "Error"
    },
    {
      "key": "CFG_PROFILE_preferEmptyValueForDiscretionary",
      "value": false
    },
    {
      "key": "CFG_PROFILE_applyDefaults",
      "value": false
    },
    {
      "key": "CFG_PROFILE_validateWithoutExpand",
      "value": false
    }
  ]
}
```

Retrieve the Configurator rules for given configurator perspective and set of criteria

This section defines an API that Retrieves an array of Configurator rules for given configurator perspective and matches below all set of criteria. Here user can provide abstract class or any concrete class to filter. But all of them must be subtypes of Cfg0AbsRule.

- list of persistent attributes and its values as saved query. Here attribute can be custom attributes or OOTB attributes.
- List of features / families in condition expression. Features / families might have referred in the expression using “AND/OR/NOT”. All the features / families in this list must be in the condition expression to qualify the constraint as output list. If one or more features / families of this list is not referred in a condition expression, then constraint will not qualify as output. The condition expression may refer additional features /families compared to the input list.
- List of features / families in subject expression. Features / families might have referred in the expression using “AND/OR/NOT”. All the features / families in this list must be in the subject expression to qualify the constraint as output list. If one or more features / families of this list is not referred in a subject expression, then constraint will not qualify as output. The subject expression may refer additional features / families compared to the input list.
- Available from release TC2312.0002 patch

Returns

- ITK_ok on success
- CFG0CONFIGURATOR_invalid_configuration_perspective if input perspective is invalid
- CFG0CONFIGURATOR_Invalid_Object_Type if any member of input condition features, subject features is invalid

```
CFG0CONFIGURATOR_API extern int searchConstraints(  
tag_t perspective,          /**< (I) The perspective of a Configurator  
                             Context which holds configuration parameters such as Revision Rule,  
                             Rule Date, effectivity etc. This is optional parameter. If NULLTAG then keys are  
                             values might have Revision rule and Rule date criteria. Else will consider  
                             default values. If NULLTAG then consider the query criteria entries to filter  
                             rule objects based on given product items or all rules in the system are  
                             considered. */  
  
const std::vector< tag_t >& ruleTypes,  
                             /**< (I) List of rule object types (subclass of Cfg0AbsRule) to be filtered on.  
                             The empty vector is valid input. If input is empty and saved query has the  
                             object_type criteria entry, then filtering happens based on the saved query. If  
                             saved query is also NULL then returns all rule object types */  
  
const std::vector< tag_t >& conditions,  
                             /**< (I) List of condition features or families.  
                             Empty is valid input */  
  
const std::vector< tag_t >& subjects, /**< (I) List of subject features or  
                                     families. Empty is valid input */  
const tag_t query,           /**< (I) Tag of the saved query. NULL is valid  
                             input */
```

Restricted

```

const std::map< std::string, std::string >& inputEntryMap,
                                   /**< (I) Entries map where key is property
                                   name and value is property value of
                                   the given input query */
std::vector< tag_t >& results      /**< (O) Vector of constraint objects. */
);

```

Search constraints based on Features - Example

Consider the following product context

Configurator Context	Automotive
Models:	EDI, EXI, LXI, LDI, ZXI, ZDI
Families:	
Colour	Black, Blue, PearlWhite, Red, White
Engine	Taxi, V4, V8, V8 Turbo
Seats	Bucket, Leather, Stock
Style	E-Class, Sedan HatchBack
Sunroof	T,F
Dashboard	CarbonFiber, Leather, Wooden
Transmission	Auto, Manual

Constraints:

Rules	Condition	Subject	ID
Inclusion Rule	([]ProductModels = ZXI && (([PCA]Style = E- Class [PCA]Style = Sedan)))	([PCA]Seats = Bucket_Seats) && ([PCA]Auto = Auto) && ([PCA]Sunroof = Sunroof) && ([[PCA]Rear lamp]Rear Camera = Rear Camera && [PCA]RearCamera_Cable = 4)))))	PC_PB_1709
Inclusion Rule	[]ProductModels = VDI	(([PCA]Seats = Bucket_Seats && [PCA]Color.= Red) (([PCA]Seats = Bucket_Seats && [PCA]Color.= Blue) ([PCA]Seats = Bucket_Seats && [PCA]Color.= White)))	PC_PB_11
Inclusion Rule	[PCA]Seats = Bucket_Seats	[PCA]Dashboard = CarbonFiber_Dashboard	PC_PB_404a
Exclude Rule	(([PCA]V8 = V8 && []ProductModels = ZDI)	[PCA]Auto = Auto	PC_PB_1732

Restricted

	[]ProductModels = LXI)		
--	------------------------	--	--

The Search constraints API will give the following output for the given inputs

Parameter	Example1	Example 2	Example 3	Example 4
Perspective	Automotive	Automotive	Automotive	Automotive
RuleType				
conditions		E-Class, Sedan	E-Class, Sedan	
subject	Bucket_Seats, Red, Blue		Bucket_Seats, Sunroof	
queryInput				type: 'Inclusion Rule' AND object_name like "404a"
Output				
Results	PC_PB_11	PC_PB_1709	PC_PB_1709	PC_PB_404a

Sample Code

The sample code to use the API is provided under `sample/cfg0configurator_searchconstraints_sample.cxx` file (in `sample.zip`)

Customization Scenarios

Sample code to find referenced features, families for a given rule object (`Cfg0AbsRule`)

- If any Product configurator rule object has subject or condition, then the features and families participating in the Rule will be part of the `fnd0ImpactingObjects` property of the corresponding Variant Expression Block object (`Cfg0VariantExprBlock`)
- This Variant Expression block object is a `POM_Object` (`Cfg0VariantExprBlock`) that refers to the `RuleObject` (`Cfg0AbsRule`)

- Use the [POM Enquiry APIs](#) to get the Cfg0VariantExprBlock objects where its fnd0affectedobject=RuleTag

Ref: https://docs.sw.siemens.com/documentation/external/PL20220103231146416/en-US/tcrefs/ITKFunction/group_e_n_q.html#gafc8610f6162a827477cac8de0284763b

Cfg0VariantExprBlock, (no name)

Object &00007FF4EBE2AF48 is an instance of Cfg0VariantExprBlock in file "6413968f9b88"

```

tag TAG 262879 (0x402df) Cfg0VariantExprBlock "UpsRxx6vphfuIC"
timestamp "kVgRxx6vphfuIC"
pid 3956
owning_site TAG 18182 (0x4706) POM_imc "IMC-10001"
lsd 2023/03/16 21:49:57
object_properties 0
is_traced FALSE
alsRestrictedWriteMode FALSE
is_selected FALSE
is_ghost FALSE
fnd0ImpactingObjects VLA &00007FF4EBE2B258 (length 6)
fnd0ImpactingObjects[0] TAG 86943 (0x1539f) Cfg0LiteralFamilyThread "weiNeruDphfuIC"
fnd0ImpactingObjects[1] TAG 90827 (0x162cb) Cfg0LiteralFamilyThread "AHnF_5mrphfuIC"
fnd0ImpactingObjects[2] TAG 87215 (0x154af) Cfg0LiteralFamilyThread "RqnF_5mrphfuIC"
fnd0ImpactingObjects[3] TAG 120158 (0x1d55e) Cfg0LiteralValueThread "g2sRxx6vphfuIC"
fnd0ImpactingObjects[4] TAG 118848 (0x1d040) Cfg0LiteralValueThread "AXhF_5mrphfuIC"
fnd0ImpactingObjects[5] TAG 118180 (0x1cda4) Cfg0LiteralValueThread "R2kF_5mrphfuIC"
fnd0expressions VLA &00007FF4EBE2B280 (length 1)
fnd0expressions[0] TAG 252974 (0x3dc2e) Cfg0ApplicationVariantExp "kRgRxx6vphfuIC"
fnd0affectedObject TAG 177984 (0x2b740) Cfg0ExcludeRule "00702BZH/001"
Object UID "UpsRxx6vphfuIC"
4bit Object UID "45ac64139afb968f9b88"

```

- When you get the Cfg0VariantExprBlock object, get the list of impacting objects using (fnd0ImpactingObjects) property. The list of Family and Feature thread objects are listed.
- From the Family and Feature Thread objects (Cfg0AbsFamilyThread, Cfg0AbsValueThread) find the revision Workspace object.
 - This can be done through following ITK

[SRH0_get_configured_revisions](#)

Sample Code:

Create the Collab Context:

```

tag_t context = NULLTAG;

stat = COLLABCTX_create_configuration_context ( "ConfigurationContext",
                                                "<<SOMEUNIQUESTR", "any
description", &context );

```


Get the appropriate revisionRule from ITK. This should be the same revision rule that you will be using for the [CFM get revision rules](#)

(https://docs.sw.siemens.com/documentation/external/PL20220103231146416/en-US/tcrefs/ITKFunction/group_c_f_m.html#ga6ba60c9af4bed749dfb1575c835752f8)

```
stat = COLLABCTX_configuration_set_revisionrule( context, revisionRule ); //
revisionRule is a tag_t, can be default rule
```

```
stat = SRH0_get_configured_revisions( context, noOfThreads, thread_array,
&confRevs2 );
```

- The confRevs2 will have all the appropriate revision of the Workspace object.
- To get the release status of a given workspace object, use the [WSOM ask release status](#) API (https://docs.sw.siemens.com/documentation/external/PL20220103231146416/en-US/tcrefs/ITKFunction/group_w_s_o.html#ga9097944dc2e3b2961f2a37a6167835fb)

Sample code to find the features, descriptions for each literal in the variant formula of a given SVR

Consider a formula of an SVR

[SampleCC]201-001 = FB1 & [SampleCC]100-002 = ELC & [SampleCC]100-001 = 35FT & [SampleCC]XCL-MODELS = XD

Now how do we find the map between each literal expression (**[SampleCC]201-001 = FB1**) to actual families (model family) and feature objects? The below struct has the sample code.

Usage:

For Each SVR tag create a SVR_Expression struct.

```
SVR_Expression svrExpr;

svrExpr.init( svrTag, contextTag ) // Initialize with svr and CC context tag

svrExpr.toString() // will give the report of the SVR
```

Note: The same code can be used for any variant formula (variant condition, etc) with small changes to the struct code

Restricted

```

#include <algorithm>

#include <base_utils/IFail.hxx>

#include <base_utils/ResultStatus.hxx>

#include <Cfg0configurator/cfg0configurator_errors.h>

#include <cfm/cfm.h>

#include <booleanmath/cfg_tokens.h>

#include <fnd0booleansolve/booleansolve.h>

#include <fnd0booleansolve/variant_solve_errors.h>

#include <tccore/item.h>

#include <tccore/tctype.h>

#include <tccore/tctype_deprecated.h>


// Represents a single selection in the SVR
// Eg: [NS]Color=Red

struct SVR_Selection
{
public:

    int operatorCode;

    std::string featureId;

    std::string familyId;

    std::string familyNS;

    std::string familyDesc;

    std::string featureDesc;


    std::string toString()
    {
        std::stringstream msg;

        msg << " Family: " << familyId << "(Desc: " << familyDesc << " ) "

        << " Family NS: " << familyNS

        << " " << ( operatorCode == 5 ? " = " : " != " )

        << "Feature: " << featureId << "(Desc: " << featureDesc << " ) ";

        return msg.str();
    }
};

```

```

// Each SVR tag should have an SVR_Expression struct.

// Just create a variable of struct

// SVR_Expression svrExpr;

// svrExpr.init( svrTag, contextTag ) // Initialize with svr and CC context tag

// svrExpr.toString() will give the report of the SVR

struct SVR_Expression
{
    std::string formula;

    BSLV_expression_info_t* m_expressionInfo;

    tag_t svr;

    tag_t m_contextItem;

    std::string svrName;

    std::string svrObjectDesc;

    std::string svrRuleText;


    tag_t perspective;

    tag_t revisionRule;

    std::string revRuleName;

    std::map< std::string, SVR_Selection> m_exprVsSelection;

    std::vector< SVR_Selection > m_svrSelections;

    std::vector< tag_t > familiesInExpr;

    std::vector< tag_t > allFamilies;


    // All loaded Families for the given perspective.

    std::vector<tag_t> m_allLoadedFamilies;


    // All the families in the business object config expression

    std::vector<tag_t> m_allFamiliesInExpression;


    // Value tag Vs ID map

    std::map<tag_t, std::string> m_FeatureTagVsID;

    std::map<std::string, tag_t> m_FeatureIDVsTag;

    std::map<std::string, tag_t> m_FamIDAndNSVsTag;


    // Value tag Vs Namespace map

```

```

std::map<tag_t, std::string> m_tagVsNS;

std::map<tag_t, std::vector<tag_t> > m_familyToFeaturesMap;

std::string toString()
{
    std::stringstream msg;

    msg << "\n\n";
    msg << "SVR Name:" << svrName
        << " Desc: " << svrObjectDesc
        << "\n Formula: " << svrRuleText
        << "\nRevision Rule: " << revRuleName << "\n";
    msg << "\nSelections\n";
    for ( std::vector< SVR_Selection >::iterator svrSelItr = m_svrSelections.begin(); svrSelItr != m_svrSelections.end(); ++svrSelItr )
    {

        msg << "\n" << svrSelItr->toString();
    }
    msg << "\nEnd of Selections\n";
    return msg.str();
}

void init( tag_t svr, tag_t context );

void loadAllFamilies();

void getFamilies( const BSLV_expression_info_t* expressionInfo );

std::string getFamilyNamespaceString( const tag_t& familyTag );

std::string getFamilyId( const tag_t& familyTag );

std::string getFeatureId( const tag_t& featureTag );

void loadFeatures();

void updateSVRSelections();

};

/// Loads all the families in the current perspective

// Note: This need not load all the families, we can just load the families and features of the SVR expression using the fnd0ImpactedObjects property of the
Fnd0VariantExprBlock of the Variant Rule or any variant configuration object. This code needs updation.

```

```

void SVR_Expression::loadAllFamilies()
{
    int nFamilies = 0;
    tag_t * families;

    ResultStatus stat = AOM_ask_value_tags( perspective, "cfg0OptionFamilies", &nFamilies, &families );

    int nModelFamilies = 0;
    tag_t * modelFamilies;
    stat = AOM_ask_value_tags( perspective, "cfg0ModelFamilies", &nModelFamilies, &modelFamilies );

    m_allLoadedFamilies.reserve( nFamilies + nModelFamilies );

    if ( nModelFamilies > 0 )
    {
        std::copy( &( modelFamilies.get() [ 0 ] ), &( modelFamilies.get() [ nModelFamilies ] ),
            std::back_inserter( m_allLoadedFamilies ) );
    }

    if ( nFamilies > 0 )
    {
        std::copy( &( families.get() [ 0 ] ), &( families.get() [ nFamilies ] ),
            std::back_inserter( m_allLoadedFamilies ) );
    }

    int n_loadedFamilies = 0;
    tag_t * loadedFamilies;
    POM_load_instances_possible( m_allLoadedFamilies.size(), &m_allLoadedFamilies [ 0 ], &n_loadedFamilies, &loadedFamilies );
}

std::string SVR_Expression::getFamilyNamespaceString( const tag_t& familyTag )
{
    std::string familyNamespaceString;

    std::map<tag_t, std::string>::const_iterator tagVsNSIt = m_tagVsNS.find( familyTag );
    if ( tagVsNSIt == m_tagVsNS.end() )
    {
        char * familyNamespace;
    }
}

```

```

        ResultStatus stat = AOM_ask_value_string( familyTag, "cfg0FamilyNamespace", &familyNamespace );

        familyNamespaceString = familyNamespace.getString();

        m_tagVsNS.insert( std::map<tag_t, std::string>::value_type( familyTag, familyNamespaceString ) );

    }

    else

    {

        familyNamespaceString = tagVsNSIt->second;

    }

    return familyNamespaceString;

}

// For the given value tag, get the id (
std::string SVR_Expression::getFeatureId( const tag_t& featureTag )

{

    std::string valuelString;

    std::map<tag_t, std::string>::const_iterator tagVsIDIt = m_FeatureTagVsID.find( featureTag );

    if ( tagVsIDIt == m_FeatureTagVsID.end() )

    {

        char * valuel;

        ResultStatus stat = AOM_ask_value_string( featureTag, "cfg0ObjectId", &valuel );

        valuelString = valuel.getString();

        m_FeatureTagVsID.insert( std::map<tag_t, std::string>::value_type( featureTag, valuelString ) );

    }

    else

    {

        valuelString = tagVsIDIt->second;

    }

    return valuelString;

}

std::string SVR_Expression::getFamilyId( const tag_t& familyTag )

{

    return getFeatureId( familyTag );

} // std::string Teamcenter::BusinessObjectConfigExpression::getFamilyId( const tag_t & familyTag )

// Given a SVR and context

```

```

// Gets the information from the SVR and context

// For the SVR, loads the SVR_Expression struct with data.

void SVR_Expression::init( tag_t svr, tag_t context )
{
    this->svr = svr;

    this->m_contextItem = context;

    AOM_ask_value_tag( svr, "cfg0ConfigPerspective", &perspective );

    // Get the revision rule form the
    AOM_ask_value_tag( perspective, "cfg0RevisionRule", &revisionRule );

    // Get the Revision rule id and name
    char * sConfigRevValue;

    AOM_ask_value_string( revisionRule, "current_name", &sConfigRevValue );

    revRuleName = sConfigRevValue.get();

    char * actualFormula;

    ResultStatus stat = BSLV_get_variant_expression( revisionRule, svr, &actualFormula );

    formula = actualFormula.get();

    char * svRObjectName;

    char * svRObjectDesc;

    char * sVRRuleText;

    stat = AOM_ask_value_string( svr, "object_name", &svRObjectName );

    svrName = svRObjectName.get();

    //Asking the Variant Rule Description
    AOM_ask_value_string( svr, "object_desc", &svRObjectDesc );

    svRObjectDesc = svRObjectDesc.get();

    //Asking the Variant Rule Text
    AOM_ask_value_string( svr, "cfg0VariantRuleText", &sVRRuleText );

    svrRuleText = sVRRuleText.get();

    BSLV_expression_info_t* expressionInfo;

    stat = BSLV_convert_variant_formula_to_expression( m_contextItem, revisionRule, formula.c_str(), &expressionInfo );

    loadAllFamilies();

    getFamilies( expressionInfo );

    loadFeatures();

    updateSVRSelections();
}

```

```

}

void SVR_Expression::updateSVRSelections()
{
    for ( int index = 0; index < m_svrSelections.size(); ++index )
    {
        // Get the featureTag for ID
        if ( m_FeatureIDVsTag.find( m_svrSelections[index].featureId ) != m_FeatureIDVsTag.end() )
        {
            tag_t featureTag = m_FeatureIDVsTag.find( m_svrSelections [ index ].featureId )->second;

            char * objDesc;

            AOM_ask_value_string( featureTag, "object_desc", &objDesc );

            m_svrSelections [ index ].featureDesc = objDesc.get();
        }
    }
}

// After the families are loaded, get all the features for the given families in (m_allFamiliesInExpression)
// Prepare the map of featureId vs tag to be used later.
void SVR_Expression::loadFeatures()
{
    //Get all the option values for each family using ITK CFG0_get_option_values

    int nValues = 0;

    scoped_smptr<CFG0_option_family_values_t> optionFamilyValues;

    if ( !m_allFamiliesInExpression.empty() )
    {
        ResultStatus stat = CFG0_get_option_values( perspective, m_allFamiliesInExpression.size(), &m_allFamiliesInExpression [ 0 ], &nValues,
        &optionFamilyValues );
    }

    std::vector<tag_t> allFeatures;

    //Creating map of family to option values from the ITK response

    for ( int inx = 0; inx < nValues; ++inx )
    {
        std::vector<tag_t> optionValues;

        int nValueRevisions = optionFamilyValues.get() [ inx ].n_value_revisions;
    }
}

```



```

std::copy( &(amp; optionFamilyValues.get() [inx].value_revisions [0]), &(amp; optionFamilyValues.get() [inx].value_revisions [nValueRevisions]),
std::back_inserter( optionValues ) );

if ( !optionValues.empty() )
{
    const tag_t& family = optionFamilyValues.get() [inx].family_revision;
    m_familyToFeaturesMap [family] = optionValues;

    std::copy( &(amp; optionFamilyValues.get() [inx].value_revisions [0]), &(amp; optionFamilyValues.get() [inx].value_revisions [nValueRevisions]),
std::back_inserter( allFeatures ) );
}
} // for( int inx = 0 ; inx < nValues; ++inx )

int n_loadedValues = 0;
tag_t * loadedValues;
POM_load_instances_possible( (int) allFeatures.size(), &allFeatures [0], &n_loadedValues, &loadedValues );
for ( std::vector<tag_t>::const_iterator featuresItr = allFeatures.begin(); featuresItr != allFeatures.end(); ++featuresItr )
{
    std::string featureId = getFeatureId( *featuresItr );
    m_FeatureIDVsTag.insert( std::map<std::string, tag_t>::value_type( featureId, *featuresItr ) );
}
}

// For the given expression, traverses the expression binary tree and populates the selections
// SVR_Selections list
// Also populates the families in the expression
void SVR_Expression::getFamilies( const BSLV_expression_info_t* expressionInfo )
{
    ResultStatus stat;

    // To avoid tcserver crash when 'expressionInfo' is null.
    // E.g. We get 'expressionInfo' null in following scenario
    // Initial custom variant rule have empty variant expression. In this scenario 'expressionInfo' is NULL
    if ( expressionInfo == NULL )
    {
        return;
    }
}

```

```

}

if ( expressionInfo->left != NULL )
{
    getFamilies( expressionInfo->left );
}

if ( expressionInfo->right != NULL )
{
    getFamilies( expressionInfo->right );
}

if ( expressionInfo->value != NULL )
{
    // Create SVR_Selection

    std::stringstream expressionStr;

    expressionStr << "[" << expressionInfo->value->family->family_namespace << "]" << expressionInfo->value->family->family_id

        << ( expressionInfo->operator_code == 5 ? "=" : "!=" )

        << expressionInfo->value->value_id;

    if ( m_exprVsSelection.find( expressionStr.str() ) == m_exprVsSelection.end() )
    {
        SVR_Selection svrSelection;

        svrSelection.familyId = expressionInfo->value->family->family_id;

        svrSelection.familyNS = expressionInfo->value->family->family_namespace;

        svrSelection.featureId = expressionInfo->value->value_id;

        svrSelection.operatorCode = expressionInfo->operator_code;

        m_exprVsSelection[ expressionStr.str() ] = svrSelection;

        m_svrSelections.push_back( svrSelection );
    }

    const char* exprFamilyID = expressionInfo->value->family->family_id;

    const char* exprFamilyNamespace = expressionInfo->value->family->family_namespace;

    for ( std::vector<tag_t>::const_iterator allFamiliesItr = m_allLoadedFamilies.begin(); allFamiliesItr != m_allLoadedFamilies.end(); ++allFamiliesItr )
    {
        const tag_t& familyTag = *allFamiliesItr;
    }
}

```

```

// family tag is not in allFamiliesInExpression. Then check if family tag is in expressionInfo and add to allFamiliesInExpression.

if ( std::find( m_allFamiliesInExpression.begin(), m_allFamiliesInExpression.end(), familyTag ) == m_allFamiliesInExpression.end() )
{
    const std::string& familyNameSpaceString = getFamilyNamespaceString( familyTag );

    const std::string& familyIDString = getFamilyID( familyTag );

    // Check for boolean type
    if ( !tc_strcmp( familyNameSpaceString.c_str(), exprFamilyNamespace ) && !tc_strcmp( familyIDString.c_str(), exprFamilyID ) )
    {
        std::stringstream famIdAndNS;

        famIdAndNS << "[" << familyNameSpaceString << "]" << familyIDString;

        m_FamIDAndNSVsTag [ famIdAndNS.str() ] = familyTag;

        m_allFamiliesInExpression.push_back( familyTag );
    }
}
}
}
}

```

Example:

Expression:

[SampleCC]201-001 = FB1 & [SampleCC]100-002 = ELC & [SampleCC]100-001 = 35FT & [SampleCC]XCL-MODELS = XD

SampleCC-SampleCC > SVR_SampleCC			
Any Status; Working (Modified) No Effectivity 12-Sep-2022 14:06 (Latest)			
Models	XCL-MODELS		
100	100-001	✓ XD	
		XDE	
		✓ 35FT	LENGTH
	100-002	40FT	35FT
		60FT	40FT
			60FT
201	201-001	CNG	PROPULSION
		DIE	
		✓ ELC	
	201-001	HYB	
		✓ FB1	FRONT BUMPER
		FB2	Front Bumper #1
		FB3	Front Bumper #2
			Front Bumper #3

Expression info struct after using [BSLV convert variant formula to expression](#) ITK

```

// [SampleCC]XCL-MODELS
BSLV_option_family_info_t model_family;
model_family.family_id = "XCL-MODELS";
model_family.family_namespace = "SampleCC";

// XD
BSLV_option_value_info_t xd_model;
xd_model.family = &model_family;
xd_model.value_id = "XD";

// [SampleCC]201-001
BSLV_option_family_info_t front_bumper_family;
front_bumper_family.family_id = "201-001";
front_bumper_family.family_namespace = "SampleCC";

// FB1
BSLV_option_value_info_t bumper_FB1;
bumper_FB1.family = &front_bumper_family;
bumper_FB1.value_id = "FB1";

// [SampleCC]XCL-MODELS = XD
BSLV_expression_info_t modelExpr;
modelExpr.operator_code = CFG_operator_is_equal;
modelExpr.value = &xd_model;
modelExpr.left = 0;
modelExpr.right = 0;

// [SampleCC]201-001 = FB1
BSLV_expression_info_t frontBumperExpr;
frontBumperExpr.operator_code = CFG_operator_is_equal;
frontBumperExpr.value = &color_red;
frontBumperExpr.left = 0;
frontBumperExpr.right = 0;

// ( [SampleCC]XCL-MODELS = XD ) - Add braces

```

```
BSLV_expression_info_t bracesExpr;

bracesExpr.operator_code = CFG_operator_brackets;

bracesExpr.left = &modelExpr;

bracesExpr.right = 0;


// [SampleCC]XCL-MODELS = XD ) & [SampleCC]201-001 = FB1
BSLV_expression_info_t finalExpr;

finalExpr.operator_code = CFG_operator_and;

finalExpr.value = 0;

finalExpr.left = &bracesExpr;

finalExpr.right = &frontBumperExpr;


//...
```

Using the workflow action handlers to export the rules in CSV dataset

Reference to: [Writing your own action handler](#)

Register an action handler that creates DataSet of Text type which has CSV file of the custom format of all the Rules with expressions

Use the action handler in the workflow whose target object is the Configurator Context.

The following is the pseudo code of the action handler

- Get the target object (Cfg0ProductItem – Configurator context) Item from the arguments
- From the **Cfg0ProductItem** Item, get the Configurator perspective object using
 - [AOM_ask_value_tag](#)(configContext, "cfg0ConfigPerspective", &configPerspective)
- The perspective runtime BO has the following properties that provide access to different rules for the perspective

Business Object : Cfg0ConfiguratorPerspective

Main Properties Property Bulk Loaders Operations Operation Descriptor									
rule									
Property Name	Type	Storage Type	Inherited	Source	COTS	Referenced Type	Array	Template	
cfg0AvailabilityRules	Runtime	TypedReference		Cfg0ConfiguratorPersp		Cfg0AvailabilityRule	✓	cfg0configura	
cfg0DefaultRules	Runtime	TypedReference		Cfg0ConfiguratorPersp		Cfg0AbsDefaultRule	✓	cfg0configura	
cfg0ExceptionRules	Runtime	TypedReference		Cfg0ConfiguratorPersp		Cfg0ExceptionRule	✓	cfg0configura	
cfg0ExcludeRules	Runtime	TypedReference		Cfg0ConfiguratorPersp		Cfg0AbsExcludeRule	✓	cfg0configura	
cfg0IncludeRules	Runtime	TypedReference		Cfg0ConfiguratorPersp		Cfg0AbsIncludeRule	✓	cfg0configura	
cfg0PackageRules	Runtime	TypedReference		Cfg0ConfiguratorPersp		Cfg0PackageRule	✓	cfg0configura	

- Get the appropriate rule objects using the AOM_ask_value_tags ITK
 - Eg:

```
int nDefRules = 0;
tag_t * defRules;
ResultStatus stat = AOM_ask_value_tags( perspective, "cfg0DefaultRules", &nDefRules, &defRules);
```

- From each rule, get the appropriate properties and generate a CSV file
 - cfg0ApplicabilityCondition – Applicability expression in [TC formula format](#)
 - cfg0SubjectCondition – subject expression in TC formula format
 - cfg0ObjectId
 - cfg0AsText – Text representation
- Create a dataset of text type and attach this file as named reference
 - Refer the following ITK reference for [Dataset](#)
 - File operations ITK reference: [tcfiler.h](#)
- Relate the created Dataset to the Configurator context (using IMAN_references relation)

Restricted

Using the workflow action handlers to import the rules from CSV dataset

Reference to: [Writing your own action handler](#)

Register an action handler that creates DataSet of Text type which has CSV file of the custom format of all the Rules with expressions

Use the action handler in the workflow whose target object is the Configurator Context.

User Actions:

- Update the CSV file externally.
- Create / update a dataset in the Teamcenter with the updated CSV file.
- To add a new rule to be created, add a new row with a flag in the UID column and provide
- Related the dataset to the Configurator context (if not already related)

The following is the pseudo code of the action handler

- Get the target object (CfgOProductItem – Configurator context) Item from the arguments.
- Find the CSV dataset that is related to the configurator context. Get the file from the dataset.
- Parse the file and for each row, identify if the rule is to be updated or created.
- Validate the updated formulae (Subject and applicability conditions)
- If any of the formula is not in valid format then throw the appropriate error in a separate validation CSV Data.
- For the rows marked as new and has valid formulae, create the appropriate Rule (Based on rule type given as part of a column in the row)
- For the rows that contains valid UID, update the appropriate rule with the modified formulae (subject and applicability properties)

Export large data set in different islands using custom closure rule

Step 1:

customer having large dataset is recommended to replace the **FT_Cfg0ProductItem** closure rule with below details

in file : %TC_ROOT%\install\cfg0configurator\configSiteConsInternalClosureRules.xml

```
<ClosureRule name="FT_Cfg0ProductItem"

clauses="CLASS.Cfg0ProductItem:CLASS.Cfg0AbsAllocationThread:REFBY.cfg0ProductItem:PROCESS+TRAVERSE,

CLASS.Cfg0ProductItem:CLASS.Cfg0AbsRule:REFBY.cfg0ProductItems:PROCESS+TRAVERSE: :I,

CLASS.Cfg0ProductItem:CLASS.Cfg0AbsModelFamilyThread:REFBY.cfg0OwningProductItem:PROCESS+TRAVERSE,

CLASS.Cfg0ProductItem:CLASS.Cfg0AbsConfigModuleThread:REFBY.cfg0ParentContext:PROCESS+TRAVERSE: :I"

    comments=""
    description="Closure Rule to traverse and process the Configurator objects with Configurator Product
Item as ROOT object."
    elemId="id11"
    owning_site="#id2"
    schema_format="1"
    scope="0">
<GSIdentity elemId="id91" label="xrPzpHsTxSZa4C"/>
</ClosureRule>
```

Step 2:

import the closure rule file using tcxml_import command

command:

```
tcxml import -u=infodba -p=infodba -g=dba -scope_rules -scope_rules_mode=append -
file=%TC_ROOT%\install\cfg0configurator\configSiteConsInternalClosureRules.xml
```

Step 3:

export the data using tcxml_export utility in exported data the AbsRule and Configuration modules will be exported in different islands

Restricted

ITK References

Boolean Solve:

https://docs.plm.automation.siemens.com/data_services/resources/tc/12.2/tcrefs/custom/en_US/ITKFunction/group_boolean_solve.html#ga0430d7165b8c3ac9fc890f86730f1886

Cfg0configurator.h

https://docs.plm.automation.siemens.com/data_services/resources/tc/12.2/tcrefs/custom/en_US/ITKFunction/cfg0configurator_8h.html